



**Gobierno Bolivariano
de Venezuela**

Ministerio del Poder Popular
para la Educación Universitaria

Universidad Nacional Experimental
para las Telecomunicaciones e Informática (UNETI)



REPÚBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

UNIVERSIDAD NACIONAL EXPERIMENTAL DE LAS TELECOMUNICACIONES E INFORMÁTICA (UNETI)

PROGRAMA NACIONAL DE FORMACIÓN EN INGENIERÍA EN INFORMÁTICA

Evaluación Práctica 1. Aplicación con IONIC

Programación III: Módulo 2

ESTUDIANTE:

DOCENTE: Carlos Márquez

C.I.: V-28.653.684 [GianneFran Radomile]

Barquisimeto, Mayo de 2026



ÍNDICE GENERAL

ÍNDICE GENERAL.....	1
ÍNDICE DE ILUSTRACIONES.....	2
ENLACE AL REPOSITORIO EN GITHUB.....	3
INTRODUCCIÓN.....	4
PROCESO DE DESARROLLO.....	5
PROPÓSITO GENERAL.....	5
FASE 1: CONFIGURACIÓN INICIAL Y GENERACIÓN DE LAS	
PÁGINAS.....	5
Figura 1: Estado inicial de la aplicación con la plantilla starter	
de Ionic.....	6
Figura 2: Estructura de archivos tras la generación de las tres	
páginas independientes.....	7
FASE 2: ESTRUCTURA DE RUTAS Y SISTEMA DE DISEÑO.....	8
Figura 3: Verificación de la ruta /inicio funcional en el	
navegador.....	11
Figura 4: Verificación de la ruta /informacion-personal	
funcional en el navegador.....	12
Figura 5: Verificación de la ruta /contacto funcional en el	
navegador.....	13
FASE 3: IMPLEMENTACIÓN DE LAS VISTAS CON GLASSMORPHISM.	
13	
Figura 6: Página de Inicio con diseño final aplicado.....	15
Figura 7: Página de Información Personal con diseño final	
aplicado.....	17
Figura 8: Página de Contacto con diseño final aplicado.....	19
Fase 4: Validación Responsiva en Dispositivos Móviles.....	19
Figura 9: Vista responsiva móvil de la Página de Inicio.....	20
Figura 10: Vista responsiva móvil de la Página de Información	
Personal.....	21
Figura 11: Vista responsiva móvil de la Página de Contacto...22	
CONCLUSIONES.....	24
REFERENCIAS.....	25



ÍNDICE DE ILUSTRACIONES

Figura 1: Estado inicial de la aplicación con la plantilla starter de Ionic.....	6
Figura 2: Estructura de archivos tras la generación de las tres páginas independientes.....	7
Figura 3: Verificación de la ruta /inicio funcional en el navegador.....	11
Figura 4: Verificación de la ruta /informacion-personal funcional en el navegador.....	12
Figura 5: Verificación de la ruta /contacto funcional en el navegador.....	13
Figura 6: Página de Inicio con diseño final aplicado.....	15
Figura 7: Página de Información Personal con diseño final aplicado.....	17
Figura 8: Página de Contacto con diseño final aplicado.....	19
Figura 9: Vista responsiva móvil de la Página de Inicio.....	20
Figura 10: Vista responsiva móvil de la Página de Información Personal.....	21
Figura 11: Vista responsiva móvil de la Página de Contacto...	22



ENLACE AL REPOSITORIO EN GITHUB

https://github.com/apakuamita17/GRO_App_IONIC



INTRODUCCIÓN

El desarrollo de aplicaciones móviles modernas requiere el dominio de frameworks que permitan construir interfaces multiplataforma a partir de tecnologías web estándar. En el presente informe práctico se aborda la creación de una aplicación móvil mediante el uso de Ionic Framework integrado con Angular, tecnologías que en conjunto permiten desarrollar experiencias de usuario consistentes entre navegadores web y dispositivos móviles de forma nativa.

En ese sentido, la combinación es ideal, pues Ionic es un kit de desarrollo orientado a la construcción de aplicaciones móviles híbridas, que ofrece una colección de componentes de interfaz prefabricados que siguen las pautas de diseño de iOS y Android. Su capacidad de integración con Angular permite estructurar aplicaciones robustas mediante el patrón de módulos, componentes y servicios, mientras que Capacitor actúa como puente para empaquetar el código web hacia plataformas nativas.

A diferencia de un enfoque puramente técnico, en este programa se planteó desde el inicio la construcción de una identidad visual fuerte y diferenciadora. Para ello se definió una paleta cromática oscura basada en Deep Navy con acentos en violeta eléctrico, una tipografía moderna combinando Inter como fuente principal y Montserrat como fuente de títulos, así como un sistema de tarjetas con efecto glassmorphism que aporta profundidad visual sin recurrir a elementos decorativos saturados. En el proceso de desarrollo se hizo uso del entorno de desarrollo Antigravity de Google asistido por el modelo Gemini, elegido por su integración con Angular, herramienta perteneciente a la misma compañía.



PROCESO DE DESARROLLO

PROPÓSITO GENERAL

Para este proyecto, se solicitó la creación de una aplicación con IONIC que contenga 3 menús: Inicio, Información Personal y Contacto. Por ello, se desarrolló una aplicación móvil híbrida con un menú lateral funcional que permite al usuario navegar entre tres secciones independientes, cada una con una función y contenido visual diferenciados. A continuación, se describe el proceso completo dividido en cuatro fases consecutivas.

FASE 1: CONFIGURACIÓN INICIAL Y GENERACIÓN DE LAS PÁGINAS

El proyecto se inicializó desde la herramienta de línea de comandos de Ionic CLI con la plantilla sidemenu, la cual provee de fábrica una estructura base con menú lateral funcional. Dicha plantilla incluye una página genérica llamada “folder” que renderiza contenido según el parámetro de URL recibido, lo cual sirvió como punto de partida mas no como solución final, dado que el objetivo era contar con tres páginas verdaderamente independientes.

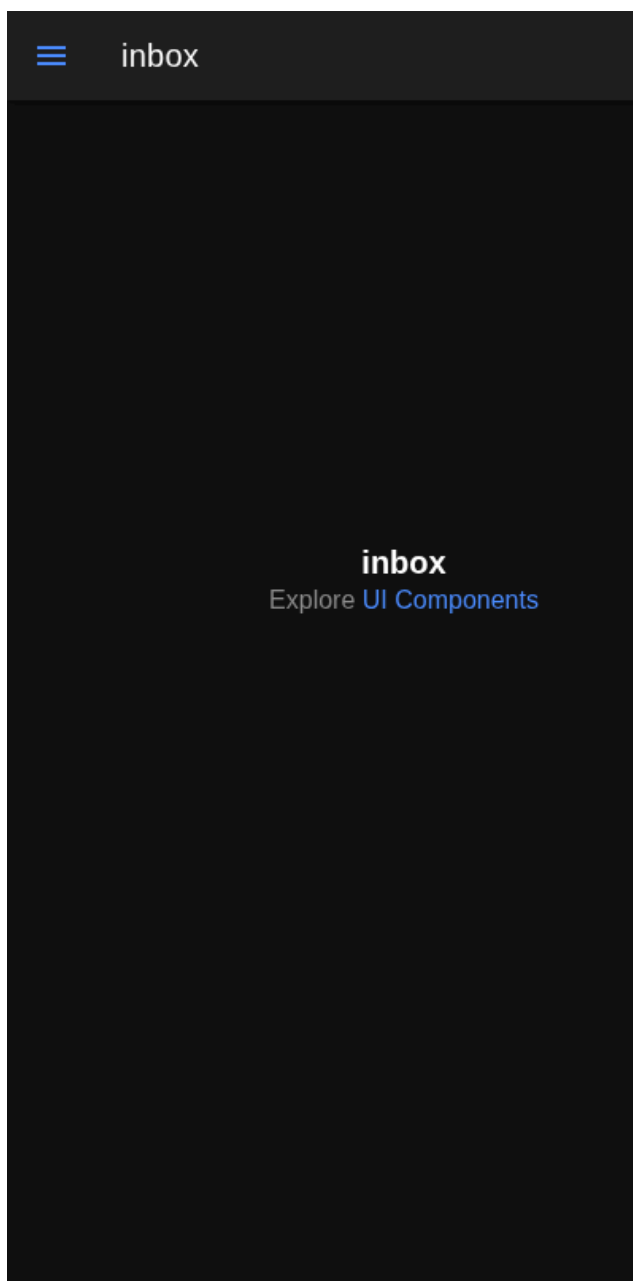


Figura 1: Estado inicial de la aplicación con la plantilla starter de Ionic.

Luego, con la misma herramienta de línea de comandos, se generaron tres páginas independientes mediante el comando `ionic generate page`, el cual crea automáticamente la estructura completa de cada página: el módulo

principal, el módulo de enrutamiento, el archivo de lógica TypeScript, la plantilla HTML, los estilos CSS y el archivo de pruebas unitarias. Los comandos ejecutados fueron: 1) ionic generate page inicio, 2) ionic generate page informacion-personal y 3) ionic generate page contacto

El resultado de estas tres ejecuciones fue la creación de tres carpetas dentro del directorio src/app/, cada una con seis archivos generados automáticamente por el CLI, estructura que permite a cada página ser un módulo autónomo, lo cual facilita el mantenimiento, la carga perezosa y la escalabilidad futura del proyecto.

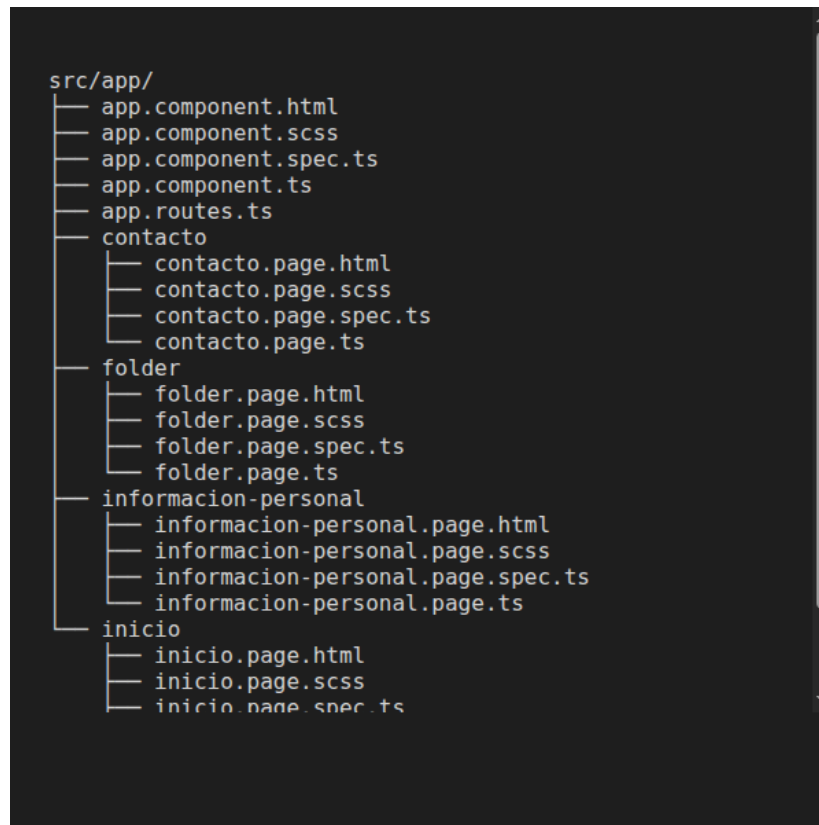


Figura 2: Estructura de archivos tras la generación de las tres páginas independientes.

Una vez generadas las páginas, se procedió a eliminar la carpeta folder que venía con la plantilla starter, para dejar al proyecto con una

arquitectura limpia donde cada sección del menú corresponde a su propia página independiente.

FASE 2: ESTRUCTURA DE RUTAS Y SISTEMA DE DISEÑO

Una de las decisiones técnicas más relevantes del proyecto fue la implementación de carga perezosa (lazy loading) para cada una de las tres rutas. Esta técnica consiste en que el código de cada página solo se descargue en la memoria del dispositivo cuando el usuario solicita visitarla, en lugar de cargarse todo de manera simultánea al iniciar la aplicación. Esto mejora significativamente el tiempo de carga inicial, especialmente en dispositivos móviles con conexiones limitadas.

La configuración del archivo `app-routing.module.ts` quedó estructurada de la siguiente manera: se definió una ruta vacía que redirige automáticamente a `/inicio` para evitar que el usuario vea una pantalla en blanco al abrir la app, y se añadieron tres rutas con carga perezosa apuntando a los módulos de cada página:

```
{ path: "", redirectTo: 'inicio', pathMatch: 'full' }
```

```
{ path: 'inicio', loadChildren: () => import('./inicio/inicio.module').then(m => m.InicioPageModule) }
```

```
{ path: 'informacion-personal', loadChildren: () => import('./informacion-personal/informacion-personal.module').then(m => m.InformacionPersonalPageModule) }
```

```
{ path: 'contacto', loadChildren: () => import('./contacto/contacto.module').then(m => m.ContactoPageModule) }
```

Paralelamente, en el archivo `app.component.ts` se actualizó el arreglo `appPages` que define los ítems del menú lateral. Cada ítem es un objeto con



tres propiedades: el título visible para la usuaria, la URL a la que navega y el icono de Ionicons que se muestra al lado.

Asimismo, una decisión arquitectónica importante fue centralizar todos los tokens de diseño (colores, sombras, bordes) como variables CSS en el archivo `src/theme/variables.scss`. Esto permite reutilizar la paleta desde cualquier componente sin repetir valores hexadecimales en múltiples archivos, y facilitar futuras modificaciones del diseño con un solo cambio de valor.

Las variables principales definidas fueron:

```
--gf-bg-primary: #0A0E27;    // Deep Navy — fondo principal
```

```
--gf-bg-secondary: #131B3D;  // Navy claro — cards elevadas
```

```
--gf-bg-glass: rgba(19, 27, 61, 0.6); // Base del glassmorphism
```

```
--gf-accent-primary: #7C3AED;    // Violeta Eléctrico — botones e  
iconos activos
```

```
--gf-accent-soft: rgba(124, 58, 237, 0.15); // Violeta suave — fondos  
sutiles
```

```
--gf-text-primary: #F5F5F7;    // Texto principal
```

```
--gf-text-secondary: #A1A1AA;  // Texto secundario
```

Adicionalmente, se cargó desde Google Fonts la tipografía Inter como fuente principal del cuerpo de texto, y Montserrat como fuente de títulos



grandes, y se aplicaron globalmente desde el archivo `src/global.scss` mediante un selector universal y reglas específicas para los encabezados.

Un detalle técnico que merece mención es que durante el desarrollo se detectó que Ionic incluye por defecto un archivo `dark.system.css` que aplica automáticamente un modo oscuro basado en las preferencias del sistema operativo. Esto entraba en conflicto con la paleta personalizada del proyecto, al sobrescribir los tokens definidos. La solución fue deshabilitar dicho archivo en `angular.json` para garantizar que el menú lateral y el resto de la aplicación mantuvieran consistentemente la paleta Deep Navy sin interferencias del modo oscuro automático.

Entonces, una vez configurado el sistema de rutas y los estilos base, se levantó el servidor de desarrollo mediante el comando `ionic serve` y se procedió a verificar manualmente que cada una de las tres rutas respondiera correctamente. A continuación, se presentan las capturas de las tres rutas funcionando con su contenido placeholder antes de la implementación del diseño final:



inicio

Figura 3: Verificación de la ruta /inicio funcional en el navegador.



Figura 4: Verificación de la ruta /informacion-personal funcional en el navegador.



Figura 5: Verificación de la ruta /contacto funcional en el navegador.

FASE 3: IMPLEMENTACIÓN DE LAS VISTAS CON GLASSMORPHISM

Esta fue la fase más extensa del proyecto, ya que involucró el diseño e implementación del contenido visual específico de cada una de las tres páginas, manteniendo coherencia con la identidad visual definida: paleta Deep Navy + Violeta Eléctrico, tipografía Inter/Montserrat, y tarjetas con efecto glassmorphism.



Para mantener la consistencia, se definió una clase utilitaria .glass-card en src/global.scss que aplica el efecto glassmorphism con los siguientes atributos: fondo semi-transparente azul-navy, desenfoque de fondo (backdrop-filter: blur(12px)), borde sutil blanco al 8%, bordes redondeados de 20px y una sombra suave con tinte violeta. Esta clase fue reutilizada en todas las tarjetas del proyecto, lo que le da un estilo visual unificado.

La página de Inicio fue diseñada como la primera vista que la usuaria observa al abrir la aplicación, por lo que su propósito principal es presentar a la autora con una identidad clara y ofrecer un acceso directo al perfil completo. Su estructura visual contiene los siguientes elementos:

- Un encabezado de Ionic con el título “Inicio” y el botón que despliega el menú lateral.
- Un título principal grande en tipografía Montserrat 800: “¡Saludos! Soy GianneFran.”
- Un subtítulo en Inter con tono secundario: “Estudiante de Ingeniería en Informática | Barquisimeto, Venezuela.”
- Un párrafo descriptivo que comunica la pasión profesional: “Apasionada por la reutilización y optimización de software y la creación de soluciones informáticas que impacten positivamente el entorno social y tecnológico.”
- Un botón principal con texto “Explorar mi Trayectoria” en color violeta eléctrico, que navega directamente a la página de Información Personal mediante el atributo routerLink de Angular.



Figura 6: Página de Inicio con diseño final aplicado.

Luego, la página de Información Personal fue concebida como una tarjeta de presentación digital donde se organiza el perfil profesional completo. Esta es la página con mayor densidad de contenido del proyecto, ya que agrupa cuatro bloques temáticos en una sola pantalla. Cada bloque se implementó como una tarjeta con la clase `.glass-card`, manteniendo separación visual mediante espaciado vertical.

Las cuatro secciones implementadas fueron:



Sección 1 — Perfil Académico: > Actualmente cursando Ingeniería en Informática en la UNETI de Venezuela, con especial interés en desarrollo full-stack y gestión de proyectos.

Sección 2 — Stack Tecnológico: Las tecnologías se presentaron como ion-chip agrupados por categoría. Los chips se personalizaron con fondo violeta suave y texto violeta eléctrico, con bordes completamente redondeados (border-radius: 999px) para reforzar la estética moderna del proyecto.

Frontend: Angular, Ionic, React Native

Backend: Django

Sección 3 — Proyecto Destacado: Se destacó el proyecto PST III (CANTV-APP), descrito como: Desarrollo de Aplicación Móvil de Gestión de Combustible en los Motogeneradores Eléctricos en CANTV Lara, Región Occidente.

Sección 4 — Otros Intereses: Una lista de intereses personales presentados como chips: Idiomas, Asistencia Virtual, Comunicación, Música, Ciencia.

Una decisión de diseño importante fue evitar la anidación de tarjetas (cards dentro de cards), un anti-patrón común que genera ruido visual sin aportar jerarquía real. En su lugar, se utilizaron secciones separadas verticalmente, lo que mantiene la información organizada sin saturar la pantalla.



Figura 7: Página de Información Personal con diseño final aplicado

La página de Contacto reúne los canales por los cuales la autora puede ser contactada. El reto en su diseño fue evitar que se sintiera como un formulario corporativo frío, por lo que se optó por un encabezado con tono cálido y una lista clara de canales acompañados por iconos de marca. Su estructura contiene:



- Encabezado de Ionic con el título “Contacto”.
- Un bloque hero con título grande “¿Hablamos?” en Montserrat 800.
- Un párrafo invitacional: “Estoy abierta a colaborar en proyectos desafiantes sobre desarrollo full-stack en ecosistemas de escritorio y móvil.”
- Una tarjeta con efecto glassmorphism que contiene una lista (ion-list) de cuatro canales de contacto, cada uno presentado como un ion-item con su icono violeta correspondiente:
 - Email: apakuamita17@proton.me (icono mail-outline)
 - WhatsApp / Celular: (+58) 412 076 0727 (icono logo-whatsapp)
 - GitHub: github.com/apakuamita17 (icono logo-github)
 - Ubicación: Barquisimeto, Edo. Lara (icono location-outline).
- Un botón grande “Enviar correo” que utiliza el atributo href="mailto:..." para abrir directamente el cliente de correo del dispositivo.

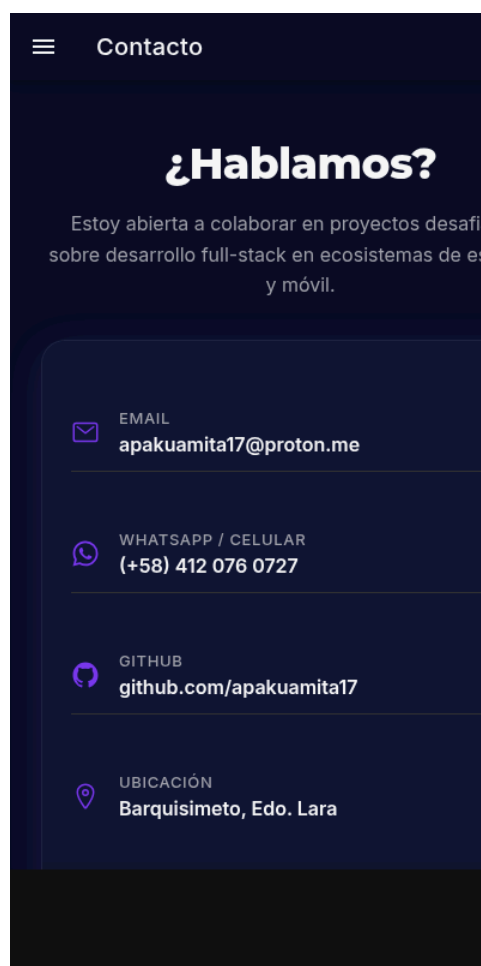


Figura 8: Página de Contacto con diseño final aplicado

Fase 4: Validación Responsiva en Dispositivos Móviles

Dado que Ionic está orientado al desarrollo de aplicaciones móviles, era imprescindible verificar que las tres páginas mantuvieran su legibilidad, proporciones y comportamiento adecuado al ser visualizadas en pantallas pequeñas. Para realizar esta validación se utilizó el modo responsivo de las herramientas de desarrollador del navegador, simulando un dispositivo iPhone estándar. A continuación, se presentan las tres capturas correspondientes a la vista móvil de cada página:



Figura 9: Vista responsiva móvil de la Página de Inicio

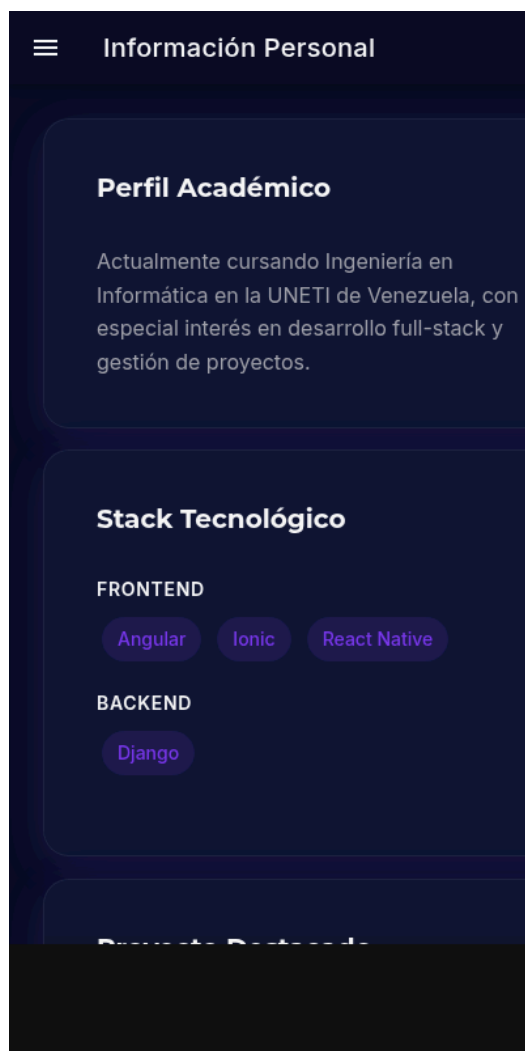


Figura 10: Vista responsiva móvil de la Página de Información Personal

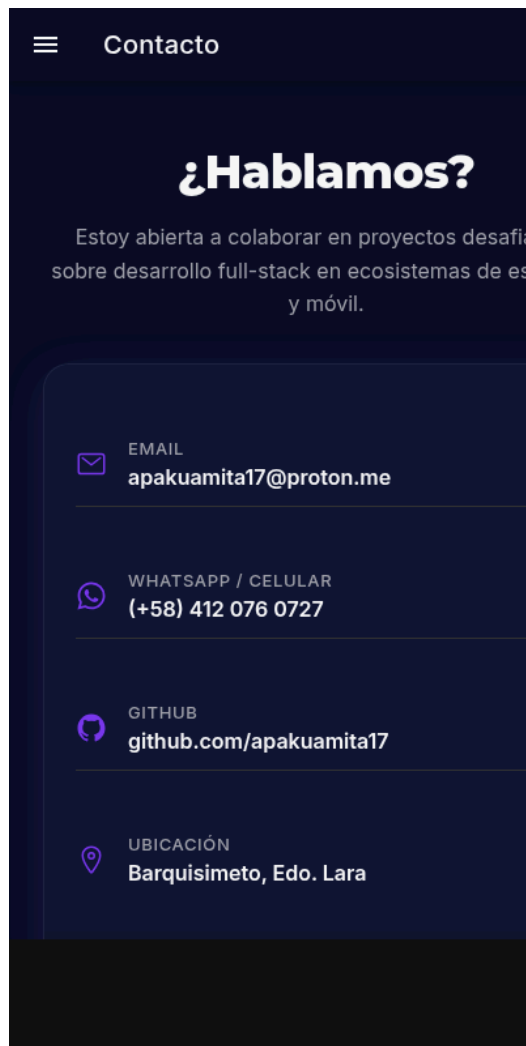


Figura 11: Vista responsiva móvil de la Página de Contacto

En las tres capturas se puede observar que el menú lateral, los componentes de Ionic y la tipografía mantienen su comportamiento adecuado en pantallas pequeñas. El espaciado se ajusta automáticamente, los chips conservan su proporción y el efecto glassmorphism se mantiene visible incluso en viewport reducido. Además, los botones y áreas interactivas mantienen un tamaño adecuado para uso táctil, lo cual confirma que los



componentes nativos de Ionic gestionan correctamente la adaptación responsiva sin necesidad de reglas CSS adicionales para los breakpoints comunes de dispositivos móviles.



CONCLUSIONES

La realización de la presente evaluación práctica permitió consolidar conocimientos fundamentales sobre el desarrollo de aplicaciones móviles híbridas mediante el uso de Ionic Framework integrado con Angular. Más allá del cumplimiento de los requisitos técnicos básicos, este proyecto planteó desde el inicio el desafío adicional de construir una identidad visual fuerte y consistente, lo cual enriqueció significativamente el proceso de aprendizaje.

La generación de páginas independientes mediante Ionic CLI demostró ser una práctica que aporta beneficios reales en términos de mantenibilidad, organización del código y rendimiento. Combinada con la técnica de carga perezosa, esta arquitectura prepara el proyecto para escalar de manera ordenada en futuras iteraciones, permitiendo añadir nuevas secciones sin reescribir la lógica de navegación existente.

La implementación del sistema de variables CSS centralizadas y la aplicación del efecto glassmorphism mediante una clase utilitaria reutilizable demostraron que un buen sistema de diseño no se trata de aplicar estilos uno por uno en cada componente, sino de definir tokens y patrones consistentes que después se reutilizan en todo el proyecto. Este enfoque facilita las modificaciones futuras y garantiza coherencia visual sin esfuerzo adicional.

Un aprendizaje técnico relevante surgió durante el proceso al detectar y resolver el conflicto entre el modo oscuro automático de Ionic (definido en `dark.system.css`) y la paleta personalizada del proyecto. Este tipo de conflictos son comunes cuando se trabaja con frameworks que traen opiniones de diseño predefinidas, y aprender a identificarlos y resolverlos forma parte esencial del oficio del desarrollo móvil.



REFERENCIAS

Google (2026). *Gemini* (versión 3.1 Pro) [Modelo de lenguaje amplio].
Disponible en: <https://gemini.google.com>